

AI Contract Auditor - Technical Exercise Write-up

Repository: <https://github.com/HindAljuaid/AI-Contract-Auditor>

Live demo: <https://ai-contract-auditor.streamlit.app>

Approach and how I measured results

I built a hybrid audit pipeline that separates language interpretation from financial decision-making. OpenAI is used to convert heterogeneous contract text into a typed contract-rule schema and, when enabled, to help resolve only unclear service-description mappings. All monetary calculations and final findings are performed deterministically in Python using integer cents, explicit rate periods, and rule-specific checks. The engine covers structural errors, unit/rate mismatches, date rules, duplicates, premiums, discounts, bundles, daily caps, exclusions, and invoice totals.

To make contract extraction more robust, I added deterministic enrichment after the language-model pass. Explicit service/rate tables are reconciled against the extracted service catalogue, cumulative volume-discount tables can be recovered when omitted, and large facility or plan-tier multiplier matrices can be parsed directly from the contract text. These enrichments are generic: they operate on contract structure rather than hospital IDs, preserve valid AI-extracted rules, and fill only missing structured information.

Hospital 1 was the only labeled dataset, so I used it strictly as a development/calibration benchmark rather than as evidence of out-of-sample performance. I evaluated all 18 error categories with precision, recall, and F1. The final H1 validator achieved 18/18 perfect categories, with 105 true positives, 0 false positives, and 0 false negatives (mean precision = 1.000, recall = 1.000, F1 = 1.000). The repository preserves the notebook, standalone validator, category evaluation, predictions, and line mappings for reproducibility.

Hospitals 2-5 are unlabeled, so I do not report accuracy for them. Instead, I monitor internal audit signals such as line-level mapping coverage, final submission confidence, flagged counts, and the manual-review queue. These are system outputs, not verified ground truth. The table below reports the final end-to-end run after the robustness improvements described above.

Hospital	Predictions	Flagged	Line mapping coverage	Avg. confidence
H2	1,125	137	95.8%	74.1%
H3	932	98	96.4%	77.3%
H4	835	156	92.7%	73.7%
H5	1,050	177	95.7%	80.4%

Where I was uncertain and why

The largest remaining source of uncertainty is ambiguous service-description mapping rather than arithmetic. Real invoice descriptions often abbreviate or paraphrase contract terminology, so unresolved or low-confidence mappings are left visible instead of being forced into a service. Robustness testing also showed that a small number of mapping changes can affect later cumulative-threshold calculations. The review queue therefore remains part of the intended workflow, not an error condition.

The extraction layer is intentionally conservative. When an explicit structured table is present, the deterministic enrichment step can recover missing rows or multiplier/discount entries; when a clause cannot be represented safely in the generic schema, it remains an unsupported rule or ambiguity and reduces confidence. Remaining examples include duplicate-service prohibitions across invoices, partial-line threshold crossings, some daily-cap aggregation scopes, and exclusion-window boundary semantics.

I also treat duplicate invoice IDs conservatively. Multiple physical source records can share one invoice ID, so the pipeline preserves source-row context, links line items to the correct source record where possible, and uses the latest physical record as the canonical submission record. Duplicate records are shown separately and are never summed together. Any unresolved duplicate context remains a review case.

For reproducibility, the final repository preserves the reviewed contract-rule JSON, service-mapping snapshots, review queues, and line-level evidence for the provided hospitals. A new or custom hospital still follows the general pipeline: fresh contract analysis, deterministic enrichment, local matching, optional AI assistance for ambiguous mappings, and deterministic auditing.

What I would do differently with another week

- Extend deterministic table extraction to PDF-native tables and additional contract layouts, with clause-level provenance for every recovered rule.
- Improve service matching with stronger contract-aware candidate generation, calibrated confidence, and reviewer-approved aliases while keeping uncertain mappings unresolved rather than guessed.
- Implement remaining unsupported semantics, especially cross-invoice duplicate-service prohibitions, partial-line cumulative-threshold crossings, daily-cap aggregation scope, and exclusion-window boundaries.
- Expand regression coverage to PDF-native and additional table layouts, and add end-to-end browser tests for the Streamlit workflow and exported artifacts.
- Use H1 only for development calibration, then add synthetic perturbation tests or a held-out validation protocol to assess confidence and mapping stability without treating H1 perfection as generalization.
- Perform structured manual review of the highest-impact H2-H5 findings, especially large expected-vs-billed differences and low-confidence mappings, before operational use.

Submission artifacts

The final submission.csv follows the repository template exactly and combines Hospitals 2-5 into one file with 3,942 unique invoice rows. The final end-to-end run flags 568 invoices (H2: 137, H3: 98, H4: 156, H5: 177). The GitHub repository contains the application, reproducible H1 validation evidence, per-hospital contract rules and audit outputs, tests, reviewed mapping snapshots for reproducibility, and the deployed Streamlit demo.